

LC-3: Longest Substring Without Repeating Characters

Question 187 Review & Code Solutions

Target Companies: Morgan Stanley, NVIDIA, Visa, BNY Mellon

Concept: Sliding Window / Two Pointers

Approach 3: Brute Force

Notes: Start from every index and extend until a duplicate is found using a frequency array.

Time Complexity: $O(N^2)$

```

class Solution {
public:
    int lengthOfLongestSubstring(string s) {
        int n = s.size();
        int ans = 0;

        // Fix starting index
        for (int i = 0; i < n; i++)
        {
            vector<int> freq(256, 0);

            // Extend ending index
            for (int j = i; j < n; j++)
            {
                // Duplicate found
                if (freq[s[j]] > 0)
                    break;

                freq[s[j]]++;

                ans = max(ans, j - i + 1);
            }
        }

        return ans;
    }
};

```

Approach 1: Sliding Window (Frequency Array)

Notes: Example: abc abc bb. As soon as another 'a' arrives, shrink the window from the left to make the characters unique again (e.g., to bcab cbb and so on...). Use a frequency array to keep track.

Time Complexity: $O(N)$ | Space Complexity: $O(1)$

```

class Solution {
public:
    int lengthOfLongestSubstring(string s) {

        int n=s.length();
        int r=0;
        int l=0;
        int ans=0;
        vector<int> freq(256,0);
        while(r<n){
            freq[s[r]]++;
            while(freq[s[r]]>1){
                freq[s[l]]--;
                l++;
            }

            ans=max(ans,r-l+1);
            r++;
        }

        return ans;
    }
};

```

Approach 2: Optimized Sliding Window (Last Seen Index)

Notes: Use a `last seen` index array. As soon as a duplicate comes (which means it already exists in the current window), update the left pointer directly: $l = \text{lastseen}[s[r]] + 1$.

Time Complexity: $O(N)$ | Space Complexity: $O(1)$

```
class Solution {
public:
    int lengthOfLongestSubstring(string s) {
        int n=s.length();
        int ans=0;
        vector<int> lastseen(256,-1);

        int l=0;
        int r=0;
        while(r<n){
            if(lastseen[s[r]]>=l){
                l=lastseen[s[r]]+1;
            }

            lastseen[s[r]]=r;
            ans=max(ans,r-l+1);
            r++;
        }

        return ans;
    }
};
```